Ders 8 Genişletilmiş Notu: Akış Diyagramı Örnekleri: Armstrong Sayısı, Doğru Denklemi ve Algoritma Analizi

Ders 8 Genişletilmiş Notu: Akış Diyagramı Örnekleri: Armstrong Sayısı, Doğru Denklemi ve Algoritma Analizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yöntemi ve Önceki Dersten Kalan Problem
- 2. Düzeltme: Kübik Sayı Değil, Armstrong Sayısı
- 3. Armstrong Sayısı Algoritmasını Birlikte Kurmak
- 4. Akış Diyagramında Kuvvet Gösterimi
- 5. Kullanıcı Hatası ve Karar Kutusunun Sadeliği
- 6. Algoritma Analizi: Kağıt Üzerinde Sınama
- 7. Daha Az İşlemle Çözülebilir mi?
- 8. Çizim Araçları ve Ders Notasyonu
- 9. Yeni Problem: İki Noktası Verilen Doğrunun Denklemi
- 10. Bilgi Nereden Gelir?
- 11. Kağıt Üzerinde Çözüm ve İlk Akış Diyagramı
- 12. Sıfıra Bölme ve Düzeltilmiş Akış Diyagramı
- 13. Çözümün Genelliği ve Tek STOP Kuralı
- 14. Bilgisayar Bilinmeyenli İfade Hesaplayamaz
- 15. Doğru Problemi Üzerine Son Sorular
- 16. Sonraki Derse Bırakılan Problem: Üçgenin Türü

1. Dersin Yöntemi ve Önceki Dersten Kalan Problem

Hoca, bundan sonraki laboratuvar derslerinin ağırlıklı olarak örnek çözerek ilerleyeceğini, konu anlatımının ise daha çok haftanın öbür dersinde yapılacağını söyledi. Bu derste önce geçen hafta sorulan algoritmik problem çözüldü, sonra yeni bir örneğe geçildi. Hocaya göre akış diyagramında görülmemiş birkaç önemli notasyon daha var. Onlar da görüldükten sonra her türlü problemi çözecek bilgiye sahip olacağız, geriye yalnız pratik kalacak.

2. Düzeltme: Kübik Sayı Değil, Armstrong Sayısı

Hoca derse, bir öğrencinin uyarısıyla bir düzeltme yaparak başladı. Önceki derste “kübik sayı” olarak adlandırılan problemin literatürdeki doğru adı **Armstrong sayısı**dır.

Kübik sayılar başka bir şeydir. Bunlar bir tam sayının küpü olan sayılardır: 8, 27, 64, 125 gibi.

Armstrong sayısı için de “basamaklarının küpleri toplamı” demek yeterli değildir, çünkü küp yalnız üç basamaklı sayılar için geçerlidir. Genel tanım şöyledir:

$$abcde\ldots = a^n + b^n + c^n + d^n + e^n + \ldots$$

Hoca buradaki n'nin neye karşılık geldiğini sordu. Cevap **basamak sayısı**dır. Yani Armstrong sayıları yalnız üç basamaklı sayılar arasında bulunmaz. Dört basamaklı bir sayıda $n = 4$ olur ve rakamların dördüncü kuvvetleri toplamının sayıya eşit olup olmadığına bakılır. Beş basamaklıda beşinci kuvvetlere bakılır. Bu derste aranan özel durum üç basamaklı sayılardır:

$$abc = a^3 + b^3 + c^3$$

Soru: Bizim yapacağımız yalnız üç basamaklı sayılar için mi geçerli? **Cevap:** Evet. Problem bugün üç basamaklı sayılar için çözülecek.

2.1 Özel Çözüm ve Genel Çözüm

Hoca, birkaç bilgi daha eklendikten sonra bu soruya geri dönüleceğini ve bu kez bütün basamak sayıları için çözülüp çözülemeyeceğine bakılacağını söyledi. Böylece bir çözümün özel hâli ile genel hâli arasındaki fark da görülmüş olacak.

> **Hocanın vurgusu:** Bilgisayar bilimlerinde algoritmik çözümlerin mümkünse genelleştirilebilir olması çok önemlidir. Özel bir problemi çözmektense o problemi de kapsayan, daha genel problemleri de çözebilen bir çözüm üretmek önemlidir.

3. Armstrong Sayısı Algoritmasını Birlikte Kurmak

Çözüm, sınıfla birlikte adım adım kuruldu. Akış START ile başlar. Kullanıcıdan girdi almadan önce ne istendiğini söylemek için bir çıkış kutusu kullanılır: `'Üç basamaklı sayı giriniz'`. Ardından giriş kutusuyla `sayı` okunur.

Sayı tek bir değer olarak okunduğu için önce basamaklarının bulunması gerekir. Bunun için bir işlem kutusu açılır ve yüzler, onlar, birler basamağı `a`, `b`, `c` değişkenlerine atanır:

```
a = sayı DIV 100
b = (sayı - (100 * a)) DIV 10
c = sayı MOD 10
```

Satırların mantığı şöyledir:

- **Yüzler basamağı:** Sayı 100'e tam bölünür.
- **Onlar basamağı:** Sayıdan yüzler kısmı ($100 \times a$) çıkarılır, kalan sayı 10'a tam bölünür.
- **Birler basamağı:** Sayının 10'a bölümünden kalan alınır.

3.1 İşlem Önceliğini Parantezle Göstermek

İkinci satır yazılırken parantez konusu tartışıldı. Çarpmanın işlem önceliği zaten vardır, yani $100 \times a$'yı paranteze almak matematiksel olarak gerekmez. Hoca yine de böyle yazmayı tercih etmediğini, işlem önceliğinin görsel olarak görülmesi için her şeyin paranteze alınması gerektiğine inandığını söyledi. Bu yüzden ifade `(sayı - (100 * a)) DIV 10` biçiminde yazıldı.

3.2 Karar Kutusu

Basamaklar bulunduktan sonra bir karar kutusuna geçilir. İlk yazımda küpler toplamı doğrudan karar kutusunun içine yazıldı:

```
a*a*a + b*b*b + c*c*c = sayı
```

Eşitlik doğruysa `'Armstrong sayı'`, değilse `'Armstrong sayı değil'` mesajı yazdırılır. Kollar birleşir ve akış STOP ile biter.

Hoca bu sırada True kolunu bu kez sola yazdı. True kolu genelde sağa yazılır, ama sola yazılmayacak diye bir kural yoktur.

> **Hocanın vurgusu:** Karar kutusunun hangi kolunun True, hangisinin False olduğu mutlaka yazılmalıdır. Kolları yazılmayan if'ler hatalı kabul edilir.

Bir öğrenci mesajların tırnaklarını hatırlattı. Mesaj yazdırıldığı için tırnak konması gerekir. Hoca hızlı yazarken atladığını söyleyip düzeltti.

4. Akış Diyagramında Kuvvet Gösterimi

Çözüm sırasında gelen soru şuydu: Karar kutusuna doğrudan `a` küp, `b` küp, `c` küp yazabilir miyiz?

Soru: Küp, şapka işaretiyle (`a^3`) yazılabilir mi? **Cevap:** Hayır, `a^3 + b^3 + c^3 = sayı` yazmamalıyız. Üzeri ifadesi özel bir ifadedir ve fonksiyonla elde edilir. Akış diyagramı kullanıyorsak mutlaka çarpım kullanırız.

Soru: İki çarpı işaretiyle (`a**3`) yazılamaz mı? **Cevap:** Hayır. Akış diyagramlarında böyle bir notasyon gösterilmeyecek.

Peki üs 3 değil de 5 ya da 8 olursa, yani çarpma n kez yapılacaksa ne olur? Hoca bunun için ileride for döngüsü kullanılacağını söyledi. Tekrar eden işler for ya da while döngüsüyle yapılır ve bu notasyon sonraki derslerde anlatılacak. Döngü, çarpma kaç defa yapılacaksa o kadar döndürüldüğünde iş kendiliğinden yapılmış olur. İleride bu tür işler `pow` gibi fonksiyonlarla da ifade edilecek. Ama akış diyagramı için en temel yol döngüdür.

5. Kullanıcı Hatası ve Karar Kutusunun Sadeliği

Hoca, ilk yazımın doğru olduğunu ama bir şeyin farklı yapılabileceğini söyleyip sınıfa sordu. Gelen cevaplardan ikisi ayrı konular açtı.

5.1 Kullanıcı Hataları

Bir öğrenci sayının gerçekten üç basamaklı olup olmadığının kontrol edilebileceğini söyledi. Hoca bunun doğru bir tespit olduğunu belirtti. Kullanıcı hatasından kaynaklanan sorunları gidermek için kontrol yapılabilir. Fakat şu aşamada kullanıcının bilgileri doğru girdiği kabul ediliyor. “Üç basamaklı sayı giriniz” dendiğinde kullanıcının üç basamaklı sayı girdiği varsayılıyor.

> **Hocanın vurgusu:** Ödevlerde bu kontroller istenecek. Sınavda ekstradan bir şey söylenmemişse kullanıcının doğru girdi verdiği varsayılabilir. Ama gerçek hayatta böyle bir varsayım yapılamaz. Kullanıcının yapacağı hatalar da hesaba katılarak program yazılmalı, tasarım yapılmalıdır.

5.2 Karar Kutusunda Yalnız Karşılaştırma

Hocanın aradığı cevabı başka bir öğrenci verdi: Küpler toplamı önce başka bir değişkene atanabilir. Böylece işlem kutusuna bir satır daha eklendi:

```
abc = a*a*a + b*b*b + c*c*c
```

Karar kutusunda ise yalnız karşılaştırma kaldı: `abc = sayı`.

> **Hocanın vurgusu:** If’in içinde mümkün olduğunca az işlem yapılmalıdır. Geri kalan işler dışarıda halledilmeli, içeride karşılaştırma yapılmalıdır.

Hoca bunun anlamını da netleştirdi. İlk yazım yanlış değildir ve bu değişiklikle daha az iş yapılmış da olmaz. Amaç görsel karışıklığı azaltmak, yazımı kolaylaştırmak ve yapılan işlerin daha derli toplu görünmesini sağlamaktır.

5.3 Algoritmanın Son Hâli

Bu noktada akış diyagramı şu adımlardan oluşur:

1. START.
2. Çıkış: `'Üç basamaklı sayı giriniz'`.
3. Giriş: `sayı`.
4. İşlem: `a = sayı DIV 100`, `b = (sayı - (100 * a)) DIV 10`, `c = sayı MOD 10`, `abc = a*a*a + b*b*b + c*c*c`.
5. Karar: `abc = sayı` mı?
6. T kolu, çıkış: `'Armstrong sayı'`.
7. F kolu, çıkış: `'Armstrong sayı değil'`.

8. İki kol birleşir.
9. STOP.

6. Algoritma Analizi: Kağıt Üzerinde Sınama

Hoca sonra şu soruyu sordu: Algoritmik olarak doğru yaptık mı? Buna bakmak için analiz gerekir. Yani tasarlanan algoritmanın istenen girdi için istenen çıktıyı gerçekten üretip üretmediğine bakılır.

Analizin nasıl yapılacağı sorulduğunda iki cevap geldi. Bir öğrenci diyagramı bir uygulamaya işlediğini, çalıştır düğmesine basıp internetten bulduğu bir sayıyla denediğini ve doğru sonuç aldığını söyledi. Hoca bunun işin arka planda kodlanması anlamına geldiğini, programı yazıp test etme zamanının da ileride geleceğini söyledi. Burada öğrenilmek istenen ise kağıt üzerinde analizdir.

Başka bir öğrenci kağıt üzerindeki yöntemi tarif etti: Tüm değişkenler bir tabloya yazılır, sonra değişkenlerin aldığı değerler tek tek izlenir. Hocaya göre karmaşık olmayan problemler için en hızlı analiz budur.

Bu algoritmanın değişkenleri `sayı`, `a`, `b`, `c` ve `abc`'dir.

6.1 Birinci Durum: 123

Verilen: `sayı` = 123.

- `a = sayı DIV 100`: $123 \text{ DIV } 100 = 1$.
- `b = (sayı - (100 * a)) DIV 10`: $123 - 100 = 23$, ardından $23 \text{ DIV } 10 = 2$.
- `c = sayı MOD 10`: $123 \text{ MOD } 10 = 3$.
- `abc`: $1 \times 1 \times 1 = 1$, $2 \times 2 \times 2 = 8$, $3 \times 3 \times 3 = 27$. Toplam $1 + 8 + 27 = 36$.
- Karar: 36 ile 123 eşit değildir. Akış False koluna gider, sonuç `'Armstrong sayı değil'` olur.

6.2 Çıktı Mesajını İyileştirmek

Hoca bu mesajın daha güzel yazılıp yazılamayacağını sordu. Bir öğrenci "Girdiğiniz sayı bir Armstrong sayısı değildir" demenin daha iyi olacağını söyledi. Hoca bunu kabul edip bir adım daha iyisini istedi.

Başka bir öğrenci mesajın önüne `abc` yazılmasını önerdi. Hoca burada uyardı: `abc` girilen sayı değil, hesaplanan sayıdır. Yazdırılması gereken `sayı` değişkenidir. Çıkış kutusuna değişken de yazılabildiği için çözüm şudur:

```
sayı, 'Armstrong sayı değildir'
```

Böylece ekranda "123 Armstrong sayı değildir" gibi daha anlamlı bir ifade görülür.

6.3 İkinci Durum: 153

Verilen: `sayı` = 153.

- `a`: $153 \text{ DIV } 100 = 1$.
- `b`: $153 - 100 = 53$, ardından $53 \text{ DIV } 10 = 5$.
- `c`: $153 \text{ MOD } 10 = 3$.
- `abc`: $1 \times 1 \times 1 = 1$, $5 \times 5 \times 5 = 125$ ve $1 + 125 = 126$. Sonra $3 \times 3 \times 3 = 27$ ve $126 + 27 = 153$.
- Karar: 153 ile 153 eşittir. Akış True koluna gider. Daha güzel ifadeyle `sayı, 'Armstrong sayıdır'` yazdırılır.

Analiz tablosu şöyledir:

sayı	a	b	c	abc	Sonuç
123	1	2	3	36	Armstrong sayı değil
153	1	5	3	153	Armstrong sayı

6.4 Kaç Durum Sınanmalı?

Biri Armstrong sayısı olan, biri olmayan iki değer sınanmış oldu. Farklı durumlar (case) için de analiz yapılabilir, ama aynı tip durumlar için çok fazla analize gerek yoktur. Hocanın örneğiyle, 123 yerine 124 yazıldığında durum değişmez. İleride algoritmanın her durumda doğru çalıştığını göstermek için belki 5, belki 8 farklı durumda analiz yapmak gerekecek. Bu problemde iki durum vardır.

7. Daha Az İşlemle Çözülebilir mi?

Hoca bu kez daha kısa ya da daha az işlemli bir çözüm olup olmadığını sordu. Bir öğrenci kendi çözümünün resmini paylaştı. Hoca önce bu çözümün yazılım aracında çizilmiş biçimi üzerinden akış ve notasyonu konuştu, sonra işlem sayısına geçti.

7.1 Akış Yönü ve Notasyon

Paylaşılan çözümde akış önce aşağı iniyor, sonra sağa geçiyor, sonra yukarı doğru gidiyordu. Hoca bu çözümün yanlış olduğunu söylemedi ve öğrencinin kullandığı aracın gereği en hızlı yolu seçtiğini belirtti. Kendi başına çalışırken böyle bir kullanıma itirazı da yoktu.

> **Hocanın vurgusu:** Aşağı inip sonra yukarı çıkan bir akışı ödevlerde ya da sınav sorularında görmek istemiyorum. Genel alışkanlık itibarıyla akış aşağı doğru olmalı. Ya da biraz önce gösterildiği gibi kollar iki yana yazılıp sonra birbirine bağlanmalıdır.

Aynı çözümde if ve giriş kutuları da farklı sembollerle gösteriliyordu. Hoca, gerçek hayatta karşımıza çıkacağı için bu farklı gösterimlerin bilinmesini istedi. Yalnız bu ders ve bölümün derslerinde bunlar kullanılmamalı, derste gösterilen notasyon kullanılmalıdır. Çok farklı notasyon da yoktur, birkaç değişik gösterim vardır.

Araç küpü `^3` ile göstermeye izin veriyordu. Hoca burada da ricasını yineledi: Bu işler dört işlemle yapılmalıdır.

7.2 İşlem Sayısı

Sınıftan gelen yorumlar farklıydı: bir tık daha fazla işlem var gibi görünüyor; birinci basamak kısaltılmış ama gerisi uzamış; işlemler tek tek yazıldığı için uzamış ama aslında pek fark yok.

Öğrencinin çözümünde birler `MOD 10` ile, yüzler `DIV 100` ile, yani bizim çözümümüzdeki gibi bulunuyordu. Fark onlar basamağındaydı. Bir öğrenci de daha önce aynı yolu önermişti: Sayının önce `MOD 100`'ünü almak, sonra sonucu 10'a bölmek. Öğrenci, araç tek satırda iki değişkene izin vermediği için çözümünün uzadığını söyledi.

Hoca, `MOD 100` kullanılsa bile aslında bir kazanç olmadığını açıkladı. Çünkü mod işlemi kendi içinde sayının tam kısmını çarpıp çıkarır ve geriye kalanla ilgilenir:

$$\text{sayı MOD } 100 = \text{sayı} - 100 \times (\text{sayı DIV } 100) \quad \text{(ara adım eklendi)}$$

Bizim yazdığımız `sayı - (100 * a)` ifadesinin aynısıdır, çünkü `a = sayı DIV 100`'dür. Yani çarpma ve çıkarma işlemi modun içinde zaten vardır. İki yazım da aynı işi yapar. Hocanın sonucu şuydu: Birler için `MOD 10`, yüzler için `DIV 100`, onlar için de `MOD 100` ve ardından 10'a bölme ya da bunun açık yazılmış karşılığı, bu işin en az işlemle yapılma biçimidir.

7.3 Hata Paylaşmanın Değeri

Hoca çözümünü paylaşıp üzerinde eleştiri yapılmasına izin veren öğrenciye teşekkür etti. Burada çözümün iyi ya da kötü olması, kimin yaptığı önemli değildir. Hocaya göre gelen çözümlerde hata olması daha da iyidir, çünkü üzerine konuşulacak ve yapılmaması gereken şeyler vurgulanacak bir bakış açısı oluşur.

8. Çizim Araçları ve Ders Notasyonu

Soru: Akış diyagramı çizebileceğimiz, önerebileceğiniz bir program var mı? **Cevap:** Bir öğrenci draw.io'yu önerdi, hoca da bir öğrencinin paylaştığı app.diagrams.net'i kullanabileceğimizi söyledi.

Soru: Bu programlar derste kullandığımız simgelere uygun mu? **Cevap:** Hayır. Bizim simge ve işaretlere uygun bir program bulmak zordur. Bu programlar yalnız çizim amacıyla kullanılabilir.

9. Yeni Problem: İki Noktası Verilen Doğrunun Denklemi

Hocanın verdiği yeni problem şuydu: **Başlangıç ve bitiş koordinatları verilen doğrusal eğrinin denklemini hesaplayan algoritmanın akış diyagramını çiziniz.** Sınıfa kağıt üzerinde denemesi için süre verilmeden önce problemle ilgili sorular alındı.

Soru: Bazı programlarda input'tan önce ayrı bir output vermek yerine mesaj tırnak içinde input'un içine yazılabiliyor. Bizim sistemimizde bu yapılabilir mi? **Cevap:** Hayır. Input ve output iki ayrı kutucukta olacak, aynı kutucukta olmayacak.

Soru: Koordinatların x ve y'lerini dört seferde ayrı ayrı mı alacağız? Virgülle alıp bölemez miyiz? **Cevap:** Bunun bir önemi yok. Dört tane girdi alınacak. Gösterim olarak aralarına virgül konup `x1, y1, x2, y2` yazılabilir, bunda bir yanlışlık yoktur.

Soru: Eğrinin doğrusal olması mı gerekiyor? Polinom olamaz mı? **Cevap:** Hocanın beklediği soru buydu. Eğrinin doğrusal yapıda, yani lineer bir denkleme sahip olduğu kabul ediliyor. İkinci ya da üçüncü dereceden bir polinom olma olasılığı olsaydı problemi çözmek için farklı bir yöntem gerekirdi. Bu yüzden hoca soru metninde eğrinin başına "doğrusal" ifadesini ekledi. Bu da bize şu denklemi verir:

$$y = ax + b$$

10. Bilgi Nereden Gelir?

Soru: Eğrinin koordinatlarını kullanıcıdan mı alacağız, yoksa bunlar verilmiş mi? **Cevap:** Hoca bunun derste her yıl tekrar tekrar sorulan bir soru olduğunu söyleyip cevabı sınıfa bıraktı. Bir öğrencinin cevabını da aynen benimsedi: Kullanıcı vermezse biz nereden, neyi, nasıl bileceğiz?

İlk etapta bilgi sağlayan tek kaynak kullanıcıdır. Birkaç istisna dışında bir girdi varsa onu kullanıcı verir.

> **Hocanın vurgusu:** Bir bilgi isteniyorsa onu kullanıcı temin edecektir. "Bunu kullanıcı mı girecek?" sorusunu umarım bir daha duymam.

Hoca ilerleyen zamanlarda başka bilgi kaynaklarının da olacağını söyleyip sınıfa sordu:

- **Dosyalar:** Bilgi bilgisayardaki bir dosyadan gelebilir.
- **Ağ:** Bilgi ağ üzerinden gelebilir. Bir web sitesinden ya da herhangi bir API üzerinden aktarılabilir.
- **Rastgele üretim:** Bir öğrencinin yazdığı gibi değerler rastgele de üretilebilir.

11. Kağıt Üzerinde Çözüm ve İlk Akış Diyagramı

Sınıfa süre verildikten sonra hoca çözümü bir öğrenciyle birlikte kurdu. Önce akış diyagramına değil, kağıt üzerinde hangi işlemlerin gerektiğine bakıldı.

11.1 Matematiksel Hazırlık

Önce eğim hesaplanır. Eğim, iki noktanın y koordinatlarının farkının x koordinatlarının farkına bölümüdür:

$$a = m = \frac{y_2 - y_1}{x_2 - x_1}$$

Öğrenci sonra noktalardan birini, örneğin (x_1, y_1)'i seçerek denklemi yazmayı önerdi:

$$y - y_1 = m(x - x_1)$$

Ardından y'yi yalnız bırakmak istedi. Hoca burada daha doğrudan bir yol gösterdi. Aslında aranan b'dir. $y = ax + b$ denkleminden:

$$b = y - ax$$

Bu ifadede y yerine y_1, x yerine x_1 konursa:

$$b = y_1 - ax_1$$

Burada a, x_1 ve y_1'in hepsi bilinir. Bu iki adımdan geçince a ve b öğrenilmiş olur, denklem de $y = ax + b$ biçiminde elde edilir.

11.2 İlk Akış Diyagramı

1. START.
2. Çıkış: `'Başlangıç ve bitiş koordinatlarını giriniz'`.
3. Giriş: `x1, y1, x2, y2`.
4. İşlem: `a = (y2 - y1) / (x2 - x1)`, `b = y1 - a * x1`.
5. Çıkış: `'Denklem=', a, 'x+', b`.
6. STOP.

12. Sıfıra Bölme ve Düzeltilmiş Akış Diyagramı

Diyagram yazılınca bir öğrenci kendi çözümünde x_1 ile x_2'nin eşit olduğu durum için bir if koyduğunu söyledi. Sebebi, eğimin tanımsız olmasıydı.

Hoca bunun güzel bir nokta olduğunu söyledi ve ekranda kırmızıyla y eksenine paralel bir doğru çizdi. Böyle bir doğruda x_2 ile x_1 birbirine eşittir. Bu durumda bölme işleminde $x_2 - x_1$ sıfır olur ve bir değer sıfıra bölünmeye çalışılır. Bilgisayarda bunun yaygın adı **division by zero** hatasıdır. Bir hata oluşturacağı için kontrol edilmesi gerekir. İlk algoritma hocanın çizdiği öbür doğrular için $ax + b$'yi verir, ama eğimin sonsuza gittiği kırmızı doğru için çalışmaz.

> **Ek bilgi (derste anlatılmadı):** Hatanın görünümü büyük olasılıkla dile ve veri türüne bağlıdır. C'de tam sayı bölmesinde sıfıra bölme programı büyük olasılıkla durdurur. `double` bölmesi ise büyük olasılıkla hata vermeden sonsuz (`inf`) ya da tanımsız (`nan`) değer üretir. Her iki durumda da sonuç doğru denklemi olmaz, bu yüzden önceden kontrol etmek gerekir.

12.1 If, İşlemden Önce mi Sonra mı?

Hoca if'in işlem kutusundan önce mi sonra mı geleceğini sordu. Öğrenci önce olmasının mantıklı olduğunu o anda fark ettiğini söyledi. Hoca da if işlem kutusundan sonra olursa pek bir önemi kalmayacağını belirtti. Bu yüzden kontrol, değerler okunduktan hemen sonra yapılır:

```
x2 = x1
```

12.2 True Kolunda Ne Yazdırılacak?

Öğrenci True kolunda `x`'i tek tırnak içinde yazdığını söyledi.

Soru: Neden x'i tek tırnak içine alıyorum? `x1` doğrudan değer olarak yazılır, ama `x` için öyle bir değer yok. Yanlış olmaz mı? **Cevap:** Yanlış olmaz. Değişkenler işlem kutusunun içinde kullanılır. Tek tırnaklı ifade ise ekrana yazdırılan mesajdır, bu da çıkış kutusunda yapılır. Tek tırnak içindeki `x` de ekrana yazdırılan mesajın parçasıdır.

Başka bir öğrenci True kolunda doğrudan "verdiğiniz doğrunun eğimi tanımsız" gibi bir mesaj verilip bitirilmesini önerdi. Hoca "Peki, denkleme ne diyeceğiz?" diye sordu, çünkü istenen

şey denklemdir. Bir başka öğrenci denklemin zaten $x = x_2$ olduğunu söyledi. Hoca, önceki öğrencilerin de anlatmaya çalıştığının bu olduğunu belirterek bunu yazdı.

> **Ders sonrası not:** Öğrenci bu cevabı verirken $x = x_2$ için "eğimi sıfır olan denklem" dedi ve bu söz o anda düzeltilmedi. Doğrusu, derste de söylendiği gibi bu doğrunun eğimi tanımsızdır. Eğimi sıfır olan doğru, Bölüm 13'te ele alınan $y_1 = y_2$ durumudur.

Soru: Neden `x = x2` yazıyoruz? "Girdiğiniz değerler için eğim hesaplanamaz" demeyecek miyiz? **Cevap:** Biz denklemi istiyoruz. Bir öğrenci açıklamayı şöyle yaptı: $x = 5$ doğrusu, x'in 5 değeri için bütün y değerlerini alır. Yani bu da bir doğru denklemidir.

12.3 False Kolunda Yapılamayacak Olan

False kolu için bir öğrenci, çıkışa `y = a * (x - x1)` biçiminde bir ifade yazmayı önerdi. Hoca başlangıcın güzel olduğunu ama bu şekilde ilerlemenin hata olacağını söyledi. Bunun sebebi Bölüm 14'te açıklanıyor. Doğru yol, False kolunda ilk diyagramdaki işlemleri yapmaktır.

12.4 Düzeltilmiş Akış Diyagramı

1. START.
2. Çıkış: `'Başlangıç ve bitiş koordinatlarını giriniz'`.
3. Giriş: `x1, y1, x2, y2`.
4. Karar: `x2 = x1` mi?
5. T kolu, çıkış: `'x=', x2`.
6. F kolu, işlem: `a = (y2 - y1) / (x2 - x1)`, `b = y1 - a * x1`.
7. F kolu, çıkış: `a, 'x+', b`.
8. İki kol birleşir.
9. STOP.

13. Çözümün Genelliği ve Tek STOP Kuralı

Hoca, kırmızı doğru için çözümün doğru sonuç verdiğini sınıfa teyit ettirdikten sonra bir öğrenci y koordinatlarının da eşit olabileceğini söyledi. Hoca bu kez yeşille x eksenine paralel bir doğru çizip bu durumun çözülüp çözülmediğini sordu.

Bir öğrencinin cevabı şuydu: Çözüm bunu sağlıyor. $y_2 - y_1$ zaten a'yı 0 yapar. a'nın 0 olmasında da bir sakınca yoktur. Sonuçta `0x + değer` elde edilir.

Demek ki üretilen çözüm şu durumların hepsi için çalışır:

- Doğrunun x eksenine paralel olması.
- Doğrunun y eksenine paralel olması.
- Doğrunun eksenleri kesmesi.

Yani **genel bir çözüm** üretilmiş oldu. Bunun için bir tane kontrol kullanmak gerekti, çünkü x_2 ve x_1'in eşit olduğu durumda kontrol edilmezse division by zero hatası alınır.

> **Hocanın vurgusu:** Bir algoritma akışında bir tane STOP olmalıdır. Bir algoritmanın içine birden fazla STOP yazmayın, doğru olmayacaktır. Kollar en sonda birleşmeli ve algoritma orada sonlanmalıdır.

14. Bilgisayar Bilinmeyenli İfade Hesaplayamaz

Bir öğrenci başka bir çözümünün doğru olup olmadığını sordu ve çözümünü paylaştı. Çözümde $x_2 = x_1$ kontrolü vardı. False kolunda eğim `m` hesaplanıyor, sonra `ax + b` tarzı bir sonuç çıkarmak için `a` ve `b` ayrı ayrı bulunmadan şu ifade bilgisayara hesaplatılmak isteniyordu:

```
y = m * (x - x1) + y1
```

Hoca bu paylaşımın da iyi olduğunu söyledi, çünkü burada önemli bir yanılgı vardı. Bilgisayar bu işlemi yapacaktır. `m`'yi biliyor, ama `x`'i bilmiyor. Bilmediği için bu işlemi yapması mümkün değildir ve hata verecektir.

Soru: Bilgisayar bilinmeyenli denklem çıkarmıyor mu? **Cevap:** Hayır, böyle bir yeteneği yok. Bilgisayara yalnız çarpma, bölme, toplama, mod alma gibi işler ve kontrol yaptırılabilir. Bir de bu işler tekrar tekrar yaptırılabilir. Başka bir şey vermek ve "bunu hesaplasın" demek mümkün değildir.

Soru: Ne yazarsak yazalım matematiksel işlemi hesaplayan siteler var. Bunlar nasıl çalışıyor? **Cevap:** Aynı mantıkla. Arka planda biri bunun kodunu yazıyor. Biz burada nasıl algoritma tasarladıysak ve kodunu yazsak çalışacaksa, orada da ekrandaki ifadeyi bir fonksiyon çözüyor. Önemli olan, o işi yapan fonksiyonun daha önce biri tarafından dört işlem kullanılarak yazılmış olmasıdır.

> **Ek bilgi (derste anlatılmadı):** Bu tür araçlara büyük olasılıkla sembolik hesap sistemi (computer algebra system) denir. Bunlar `x`'i bellekteki bir sayı olarak değil, üzerinde kurallar uygulanan bir sembol olarak büyük olasılıkla ayrı bir veri yapısında tutar. Bu yetenek de dilin kendisinden değil, önceden yazılmış programlardan gelir.

Hoca bu örneğin herkes için yararlı olduğunu söyledi, çünkü bilgisayarın bunu çözebileceği düşünülebiliyor. Bu yüzden derste x bilinmeden hesap yapılmadı. a ve b sayı olarak bulundu, çıkışta da `a, 'x+', b` biçiminde yan yana yazıldı.

15. Doğru Problemi Üzerine Son Sorular

15.1 Yatay Doğrunun Gösterimi

Bir öğrenci y'ler eşit olduğunda True kolunu tekrar ikiye ayırmak gerekip gerekmediğini düşünmüştü. Hoca, maviyle yazılan çözüm üzerinden açıkladı: y'lerin eşit olması a'nın 0 çıkması demektir. b hesaplanıp örneğin 5 çıktığında ekrana `0x+5` yazılır. Bu da denklemin doğru olduğunu söyler, yani $y = 5$ demektir.

Bu durum, görsel olarak daha düzgün görünsün diye `y=5` yazdırmak için kontrol edilebilir. Ama burada amaç işi yapmaktır. Kullanıcıya gösterim şekliyle çok fazla ilgilenilmiyor. Önemli olan doğru sonucun ekrana yansıtılmasıdır.

15.2 Sağlama Yapmak Gerekir mi?

Soru: Denklemi sağlamak için bir girdi daha alıp bir noktayı denkleme yazabilir miyiz? **Cevap:** Hoca, soruyu doğru anladıysa buna gerek olmadığını söyledi. Burada yalnız a ve b bulunuyor ve biçimli olarak ekrana yazdırılıyor. Sağlama için x_1, y_1 ya da x_2, y_2'yi yerine koymayı gerektiren bir durum yok. Yapılan iş evrenseldir ve matematikte formülü budur. Bilgisayarın doğru mu yanlış mı hesapladığı konusunda kaygıya gerek yoktur. İnsana yaptırılsaydı sağlama düşünülebilirdi, ama bilgisayar bu konularda hata yapmaz.

> **Ek bilgi (derste anlatılmadı):** Bu söz algoritmanın mantığı için geçerlidir. Gerçek bir programda kesirli sayılar sınırlı duyarlıkla saklandığı için, hesaplanan a ve b büyük olasılıkla küçük yuvarlama farkları içerebilir. Bu yüzden kesirli sayılarla yapılan eşitlik karşılaştırmaları büyük olasılıkla beklenen sonucu her zaman vermez.

15.3 Sıfır Bölü Sıfır

Soru: Sıfır bölü sıfır olursa, yani hem x'ler hem y'ler eşitse denklem ne olur? **Cevap:** Hoca soruyu sınıfa sordu. Bir öğrenci, sıfır bölü sıfır için iki noktanın aynı koordinatlarla girilmesi gerektiğini, oysa kullanıcıdan başlangıç ve bitiş noktası istendiğini söyledi. Hoca bunu onayladı. İki aynı koordinat tek bir noktaya denk gelir ve o zaman bir eğriden bahsedilemez.

Kullanıcı elbette yanlış bilgi verebilir. Bunu kontrol edip sonucunu yazdırmak da gerekebilir. Ama soruda "başlangıç ve bitiş koordinatları verilen doğrusal eğri" dendiği için kullanıcının bilgileri şu anda doğru kabul ediliyor.

> **Ders sonrası not:** Derste bu konuşma sıfır bölü sıfır üzerinden yapıldı. Ancak Bölüm 12.4'teki düzeltilmiş diyagrama göre iki nokta aynı girilirse $x_2 = x_1$ koşulu True olur, bölmeye hiç gidilmez ve ekrana `x=` ile x_2 yazılır. Sıfır bölü sıfır büyük olasılıkla yalnız ilk diyagramda ortaya çıkar.

15.4 Negatif b

Soru: Çıktıda `x+` ve `b` yazıyoruz. b negatif olursa `x+-` biçiminde görünecek. Bu hata olur mu?
Cevap: Güzel bir soru. Şık durmaz, kullanıcıya gösterim şekli çok hoş olmaz, ama değer doğru görünür ve herhangi bir hata oluşturmaz. Kullanıcı ihtiyaç duyduğu bilgiyi bilgisayardan almış olur.

16. Sonraki Derse Bırakılan Problem: Üçgenin Türü

Hoca son olarak yeni bir soru verdi: **Kenar uzunlukları verilen bir üçgenin türünü ekrana yazdıran algoritmanın akış diyagramını çiziniz.**

Program çıktısı olarak neler yazdırılabileceği sınıfa soruldu. Gelen cevaplar dik üçgen, çeşitkenar, ikizkenar ve eşkenardı. Bir öğrenci, girilen değerlerle üçgen oluşturulamayacağının da bir çıktı olabileceğini söyledi. Hoca bunun doğru olduğunu ama kenar uzunluklarının ilk etapta doğru verildiğinin kabul edileceğini belirtti. Geniş açılı ve dar açılı üçgenin de söylenebileceğini, ama asıl ilgilenilenin eşkenar, ikizkenar ve çeşitkenar olduğunu ekledi. Dik üçgen kısmına da bakılabilir: Onu da yapabiliyor muyuz, ya da ekstradan ne yazmak gerekiyor?

Bu soru bir sonraki laboratuvar dersinde çözülecek. Hocaya göre hızlıca yapılabilecek bir iştir.

Dersin Toparlaması

Bu derste iki problem üzerinden akış diyagramı çizmenin ve bir algoritmayı değerlendirmenin temel alışkanlıkları görüldü. Armstrong sayısı örneğinde sayı `DIV` ve `MOD` ile basamaklarına ayrıldı, kuvvet çarpmayla yazıldı, hesap dışarıda yapılıp karar kutusunda yalnız karşılaştırma bırakıldı. Algoritmanın doğruluğu, değişkenlerin tabloya yazıldığı kağıt üzerinde analizle, biri koşulu sağlayan biri sağlamayan iki durumla gösterildi. Notasyon tarafında True ve False etiketleri, mesajlarda tek tırnak, ayrı giriş ve çıkış kutuları, yukarıdan aşağı akış ve tek STOP kuralı öne çıktı.

Doğru denklemi örneğinde ise önce kağıt üzerindeki çözüm yazıldı, sonra paydanın sıfır olabileceği durum işlemden önce kontrol edilerek çözüm genelleştirildi. Bilgisayarın yalnız bilinen değerlerle dört işlem, mod, kontrol ve bunların tekrarını yapabildiği, bu yüzden bilinmeyen x içeren bir ifadeyi hesaplayamadığı görüldü. Hoca sonraki adımları da açıkladı: for ve while döngüleri anlatılıp algoritmik notasyonlar bitirilecek, bol soru çözülecek, ardından dizilere, matrislere ve karakter gruplarına geçilecek. Armstrong sayısı problemi de ileride her basamak sayısı için genelleştirilecek.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.